

COMP2054 ADE

Lec22

Minimum Spanning Trees

Overview

- Quick recap of definitions needed for graphs
- Definition of “Minimum Spanning Tree” or “MST”
- Examples of MSTs
- Prim’s algorithm to find an MST
- Justification of why the algorithm always gives an MST, not just some sub-optimal spanning tree

Graphs Terminology 1

These should be just a reminder:

- Edges
 - Directed edges: “from A to B”
 - But edge “from B to A” might not exist
 - Undirected edges – between A and B
- Undirected graph:
 - A graph with no directed edges
- Directed graph (“digraph”)
 - some – not necessarily all – of the edges are directed

Graphs Terminology 2

- Weighted edges
 - Edges can be given a weight – some number
 - e.g. corresponding to
 - the time taken to traverse the edge
 - The cost of building the edge in infrastructure
 - The capacity of edge in a network

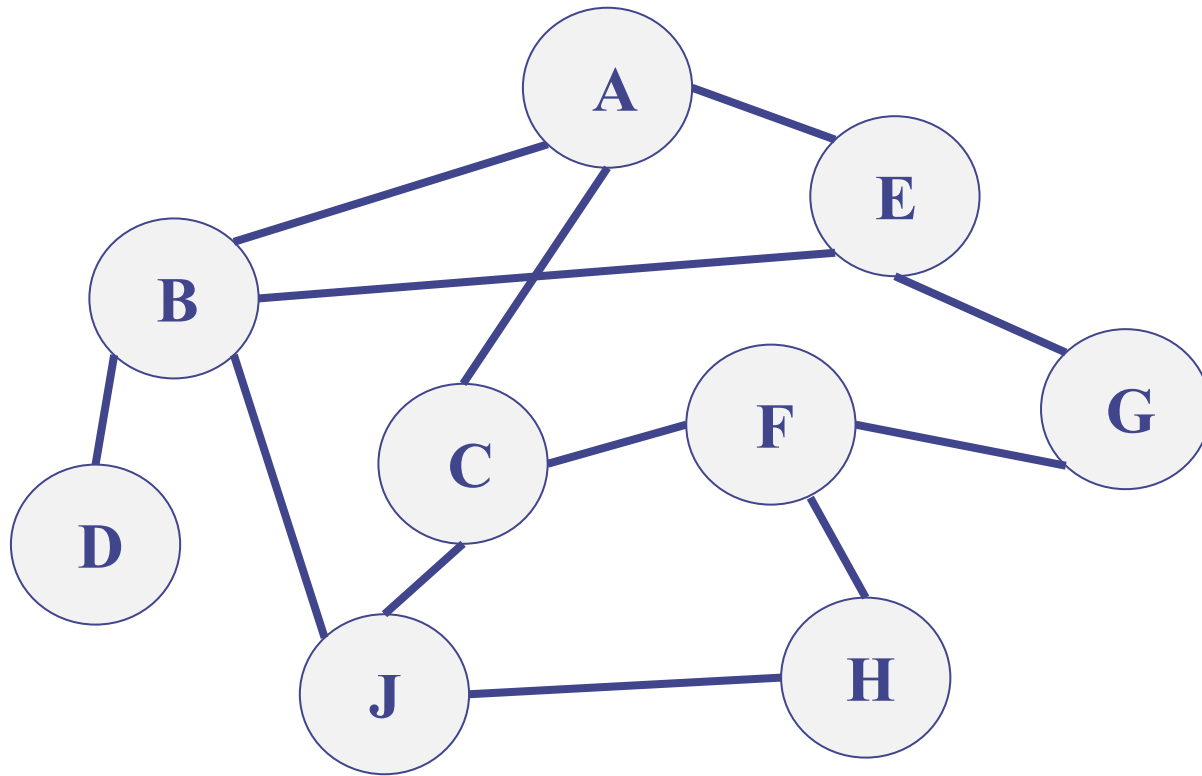
Graphs Terminology 3

- Path:
 - Sequence of edges
 - In directed graphs the edges should all go in the same direction within the path
- Connected:
 - There is a path between any two nodes
- Strongly Connected:
 - In a directed graph, there is a path from any node to any other node and respecting the directions on the edges.
- “Connected component”:
 - a subset of nodes in a graph that are internally connected, but disconnected from the rest

Spanning Tree

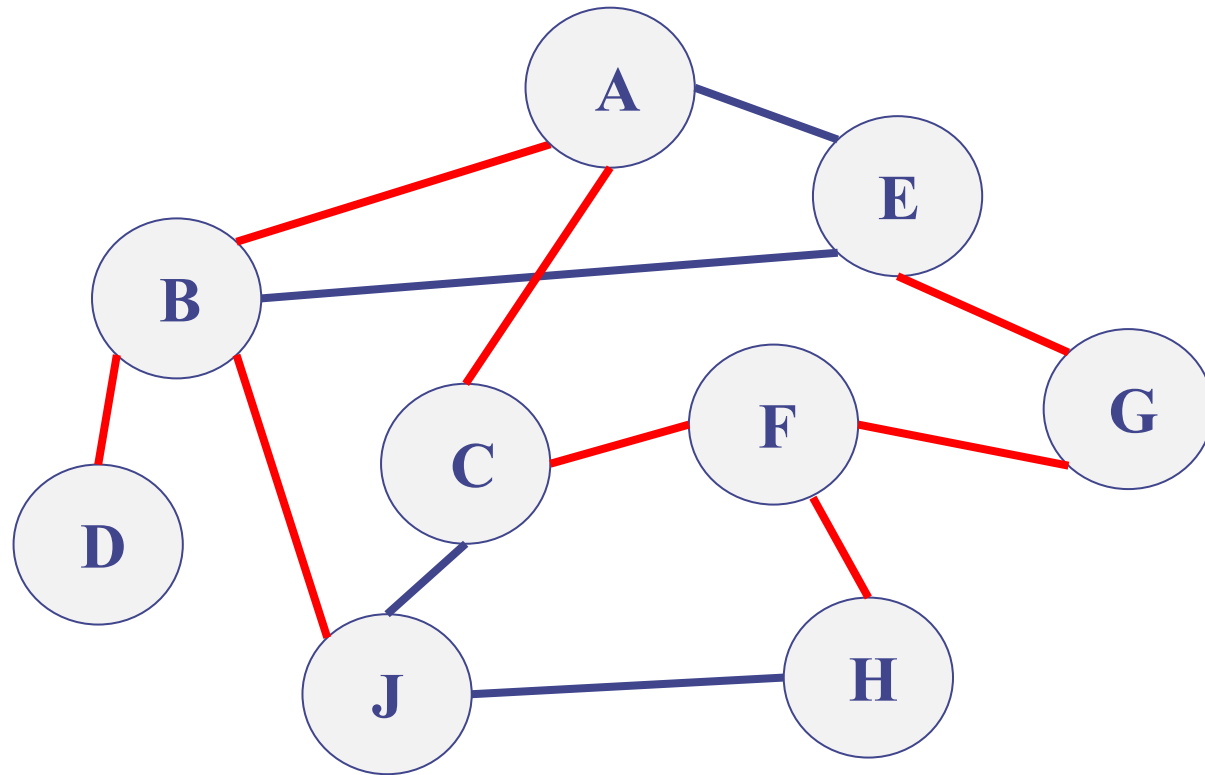
- Input: connected, undirected graph
- Output: a tree which connects all vertices in the graph, but using only the edges present in the original graph
- Note: the spanning tree is “minimal” in the sense that removing any edge means that it will no longer connects all the nodes
 - but this is a different sense than “minimum” used later

Spanning Tree: Example



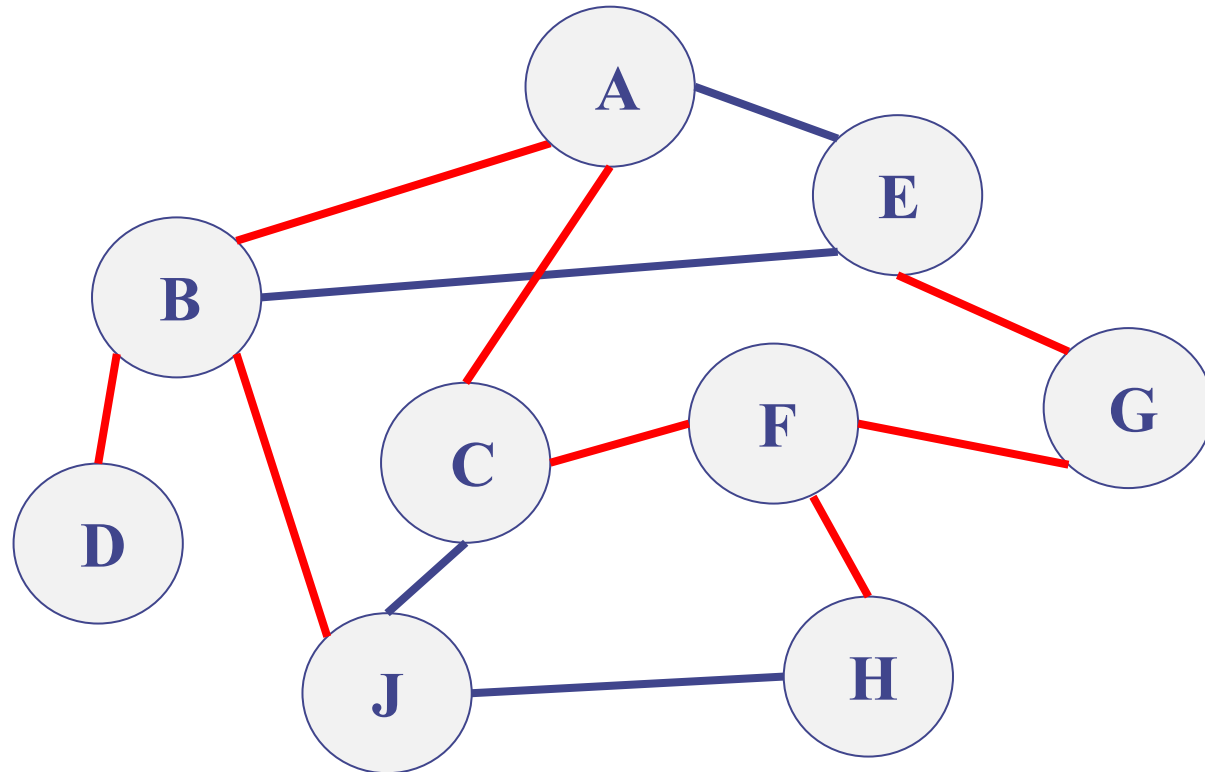
- Note the graph is connected
- A spanning tree is any subset of the edges, that form a tree and still connect all the nodes

Spanning Tree: Example (2)



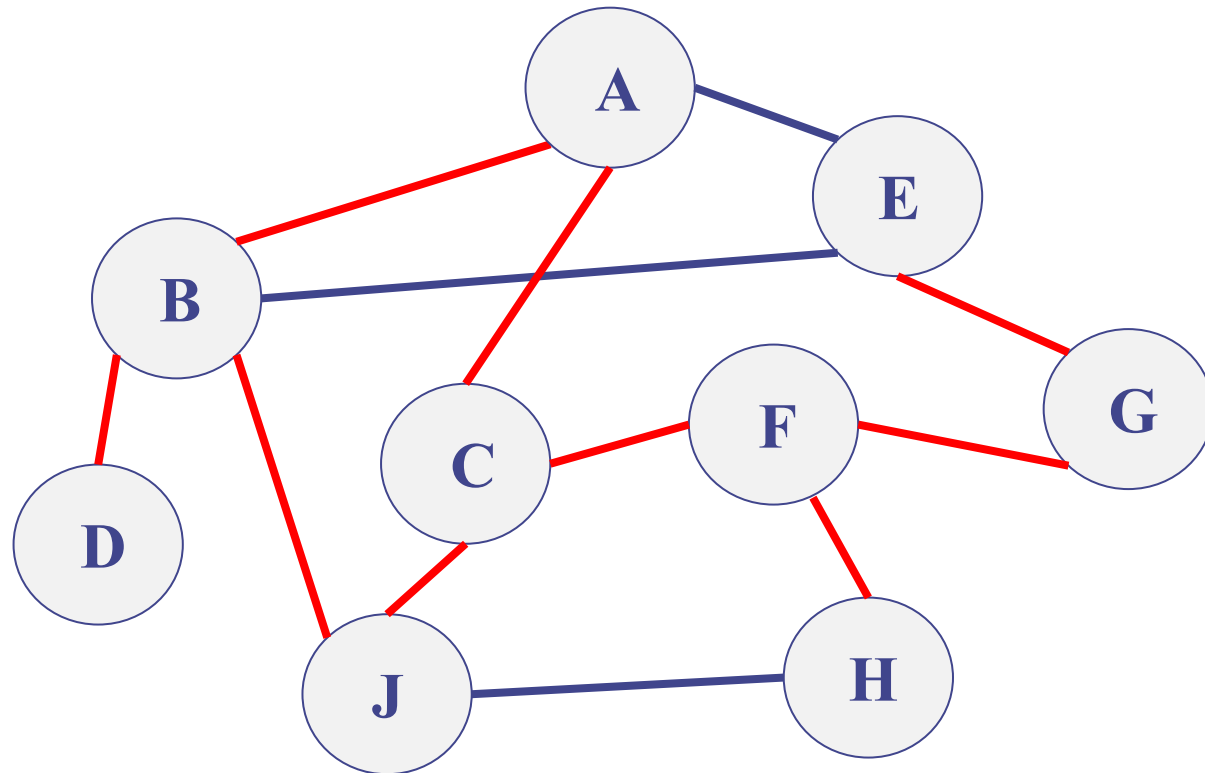
- One example, of many, is shown – in the red edges
- Note that the edge B-D has to be in all the spanning trees, but every other edge is optional in the sense that it does not have to be in the spanning tree
 - E.g. even though the example uses A-C, other spanning trees do not use A-C

Spanning Tree: Example (3)



- Within the spanning tree, there is a unique path between any two nodes
- If we remove any one edge of a spanning tree then we no longer connect all the nodes
- If we had a spanning subgraph, and that had a cycle/loop then we would be able to remove another edge:

Spanning Sub-graph: Example (4)



- Above shows a subset of edges that span the graph, but still connect all the nodes
- But it is not minimal – the edge A-C could be removed, and it would still connect all the nodes
- Removing un-needed edges always results in a spanning tree

Minimum Spanning Tree “MST”

- Input: connected, undirected, **weighted** graph
- Output: a spanning tree
 - (connects all vertices in the graph using only the edges present in the graph)
 - and is minimum in the sense that the sum of weights of the edges is the smallest possible for any spanning tree

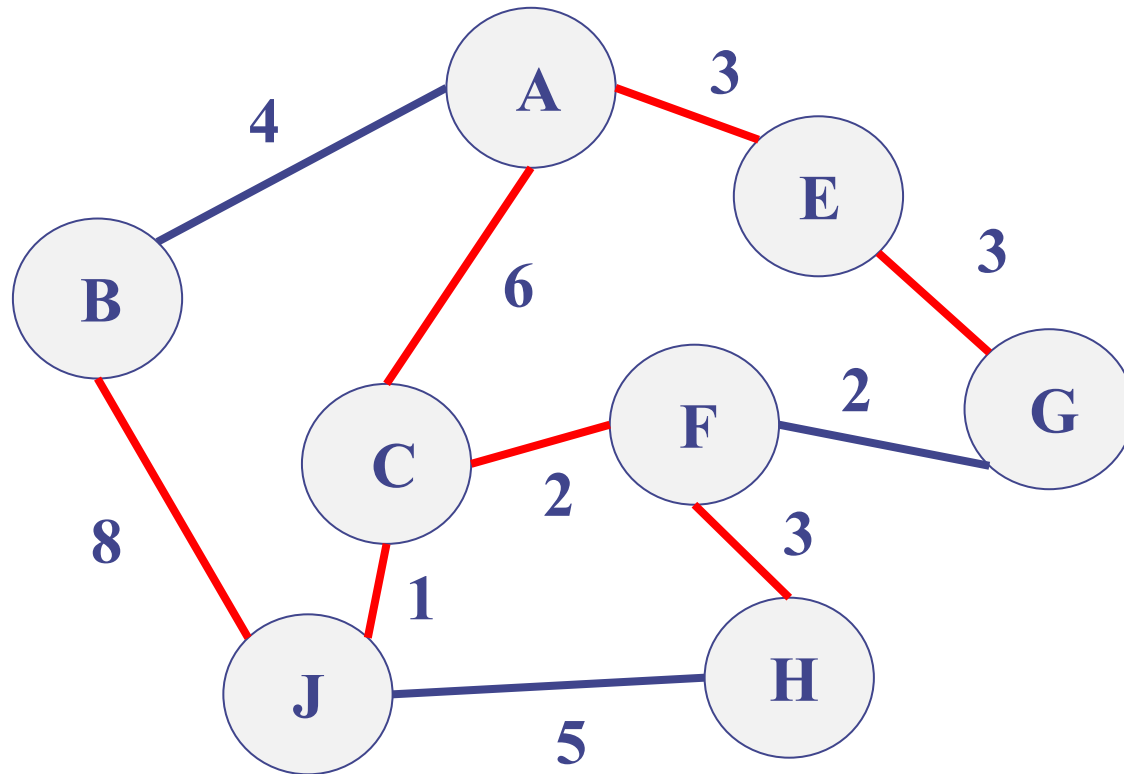
Note: “Minimal” vs “Minimum”

- Convention:
 - “minimal” generally to refer to some local property – i.e. corresponds to “locally optimal” – with respect to some allowed set of small moves
 - “minimum” refers to a global minimum
- For the MST we need a **global** minimum
 - i.e. the smallest over all possible spanning trees

MST Usages

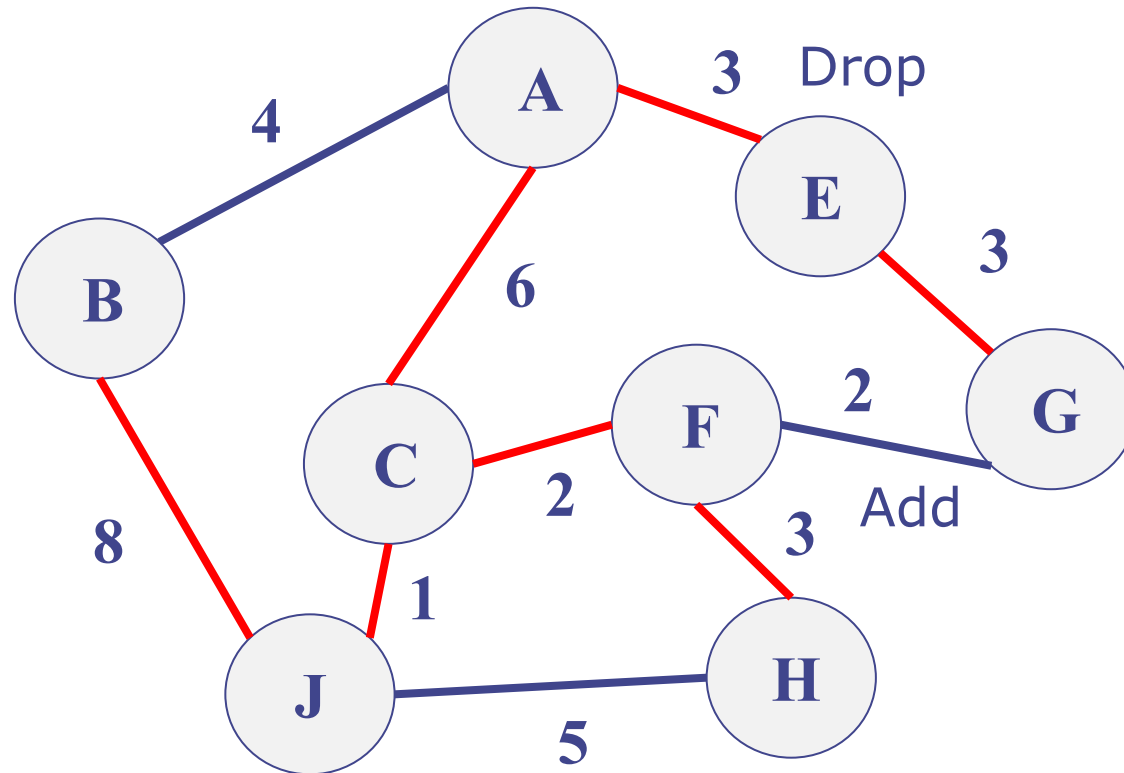
- Gas and water pipelines, optic fibres networks
 - Simply need to build a network to reach every house but minimizing total cost of construction
 - Nodes are supply/delivery locations
 - Edges are “pipes” or “cables” that we can build, and the weight is their cost
 - Aim is to build the cheapest system that meets the supply requirements
 - (In real-world might want to modify to allow redundancy in case of breakages)
- As part of other graph algorithms
 - E.g. a “Christofides algorithm” to approximately solve the TSP exploits constructing an MST and then converting to a tour https://en.wikipedia.org/wiki/Christofides_algorithm#Algorithm

Minimum Spanning Tree: 2



- The above is a spanning tree
 - We can get from any node to any other node just following the selected (red) edges
- It has weight $8+1+6+2+3+3+3 = 26$
- But is it a minimum spanning tree?
 - Exercise (online): Try to adjust the tree to reduce the total weight

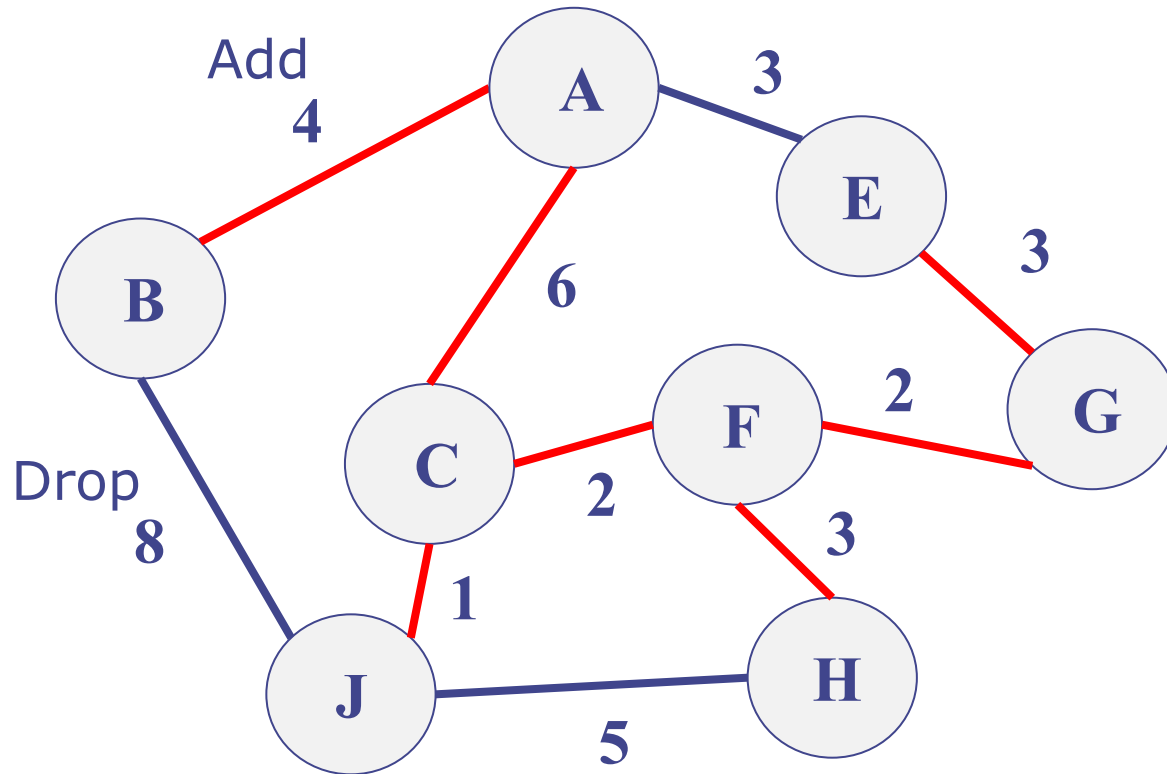
Minimum Spanning Tree: Better



Do some swaps:

- A-E to F-G : stays connected – new cost = $26 - 3 + 2 = 25$

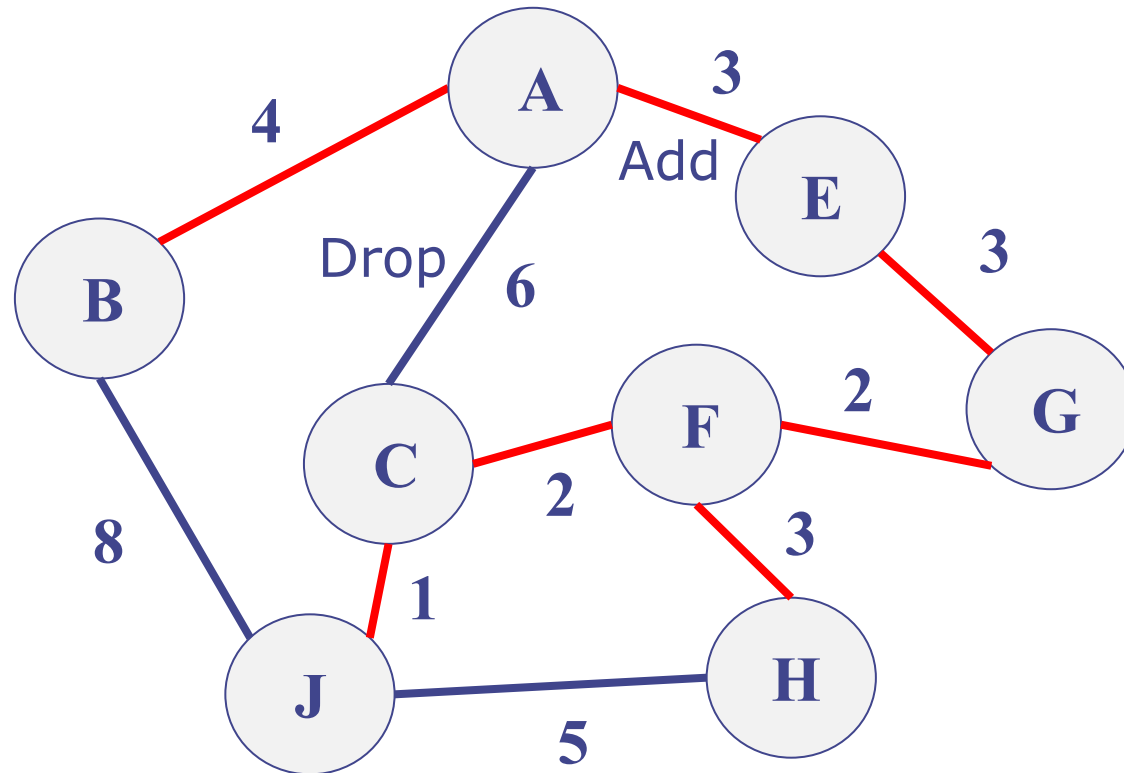
Minimum Spanning Tree: Better (2)



Then

- B-J to B-A : stays connected – new cost = $25 - 8 + 4 = 21$

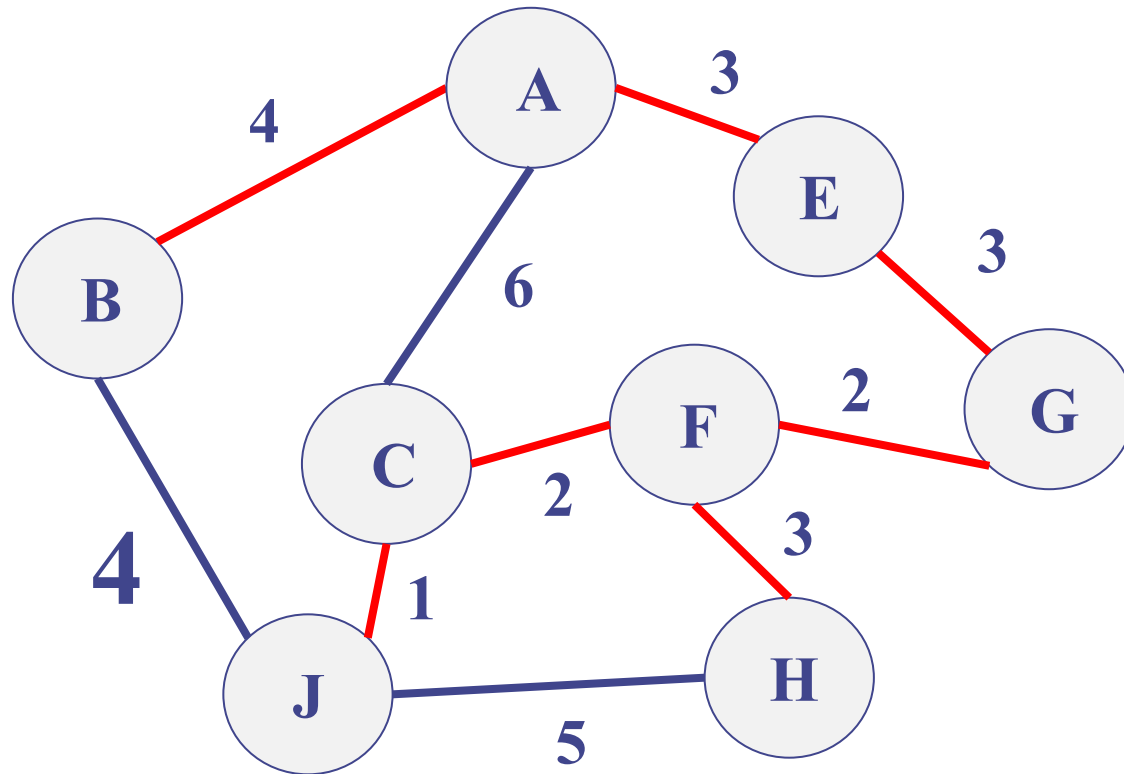
Minimum Spanning Tree: Better (3)



Then

- A-C to A-E : new cost = $21 - 6 + 3 = 18$
 - Cost = $4+3+3+2+2+3+1 = 18$
- Is this now the smallest possible !?
- Yes, it is now an MST – but how would you prove it?
- Or how to find it directly?

Minimum Spanning Tree: Not unique



- Note the changed weight of BJ
- If we swap B-A to be B-J then we get a different spanning tree, but it still has the same cost
- This shows the MST does not need to be unique

Why is an MST a tree?

- We really want a minimum spanning sub-graph
 - that is, a subset of the edges that is connected and that contains every node
- (Assuming all weights are non-negative)
If the graph has a cycle then we can remove an edge of the cycle, and the graph will still be connected, and will have a smaller weight
- If a graph is connected and acyclic then it is a tree

An MST is (generally) a TREE

- Do not confuse a minimum **TREE** with a “minimum” (shortest) **PATH**
 - Finding the shortest path that goes through all the nodes – and so connects all the nodes - is a different problem (roughly “TSP” / “Hamiltonian cycle”) from the MST (and much harder)
 - It is also different from shortest path between two nodes
- Many have people confused these on many exams, and have wrongly written something like
 - “A minimum spanning tree is a path that ...”
 - If MST meant “path” then it would say “path” 😊

How do find an MST?

- We could use “generate and test” or “brute force”:
 1. Generate all possible subsets of the edges
 2. For each subset test if it is a spanning tree
 3. Keep track of the minimum size seen
- This is very inefficient!
- With n nodes, there may be $n^2 / 2$ edges
- Giving $2^{(n^2 / 2)}$ possible subsets
- Worse than even 2^n
- At least it shows the problem can be solved
- But can we do better?

Prim's Algorithm to find an MST

- This is a greedy method
- Build up a spanning tree by
 1. Start from some node (any node is ok)
 2. Add edges one at a time until we get a spanning tree
 - add edges that keep the total weight so far as small as possible
 - so only add the edges with the smallest weights and that extend the current tree to reach a new node

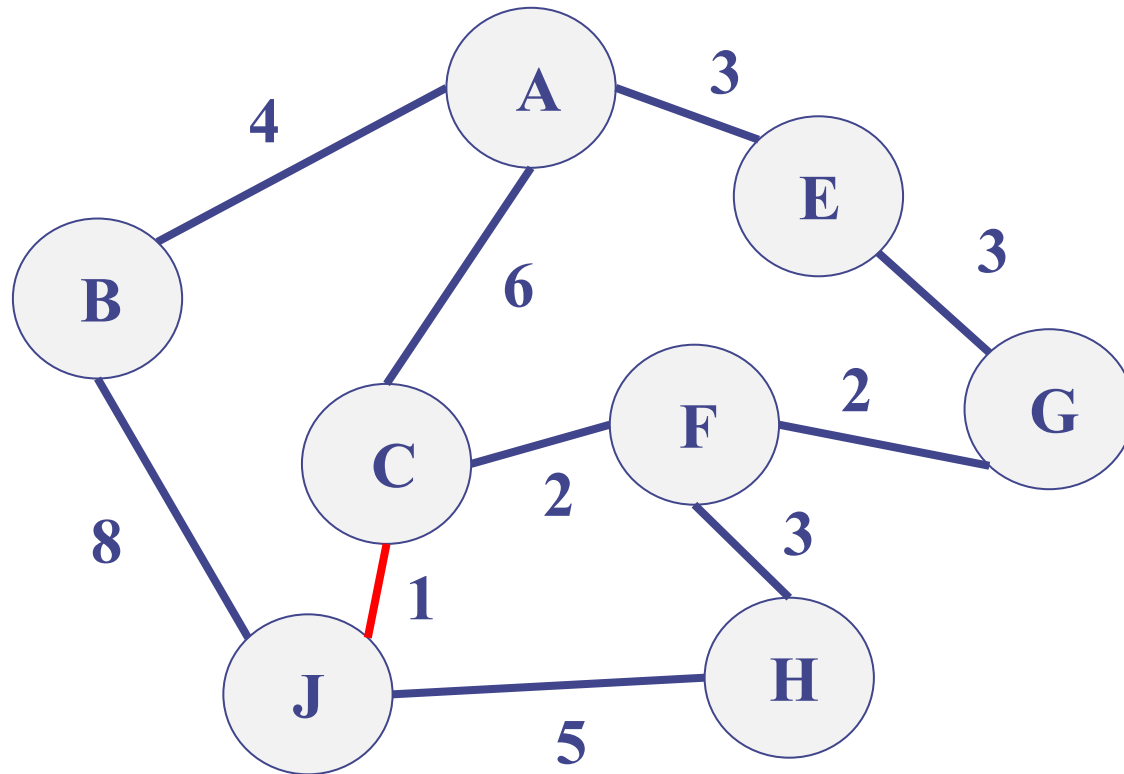
Prim's algorithm (again)

To build up an MST

Tree-so-far TSF

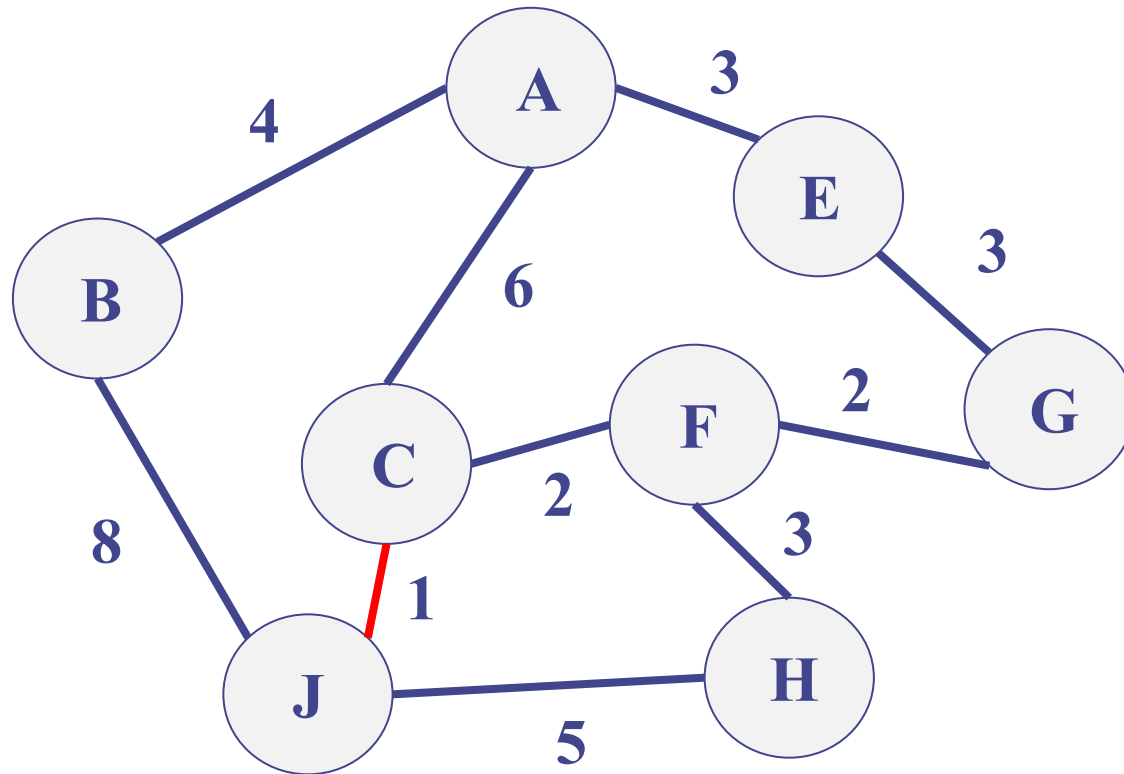
- Start by picking any vertex M , and put $TSF = \{M\}$
- Loop:
 - Add at every step, add to TSF a shortest edge from a vertex in TSF to a vertex outside, until all vertices are in TSF
 - If there are multiple shortest edges, then can take any arbitrary one

Prim's Algorithm: Example (1)



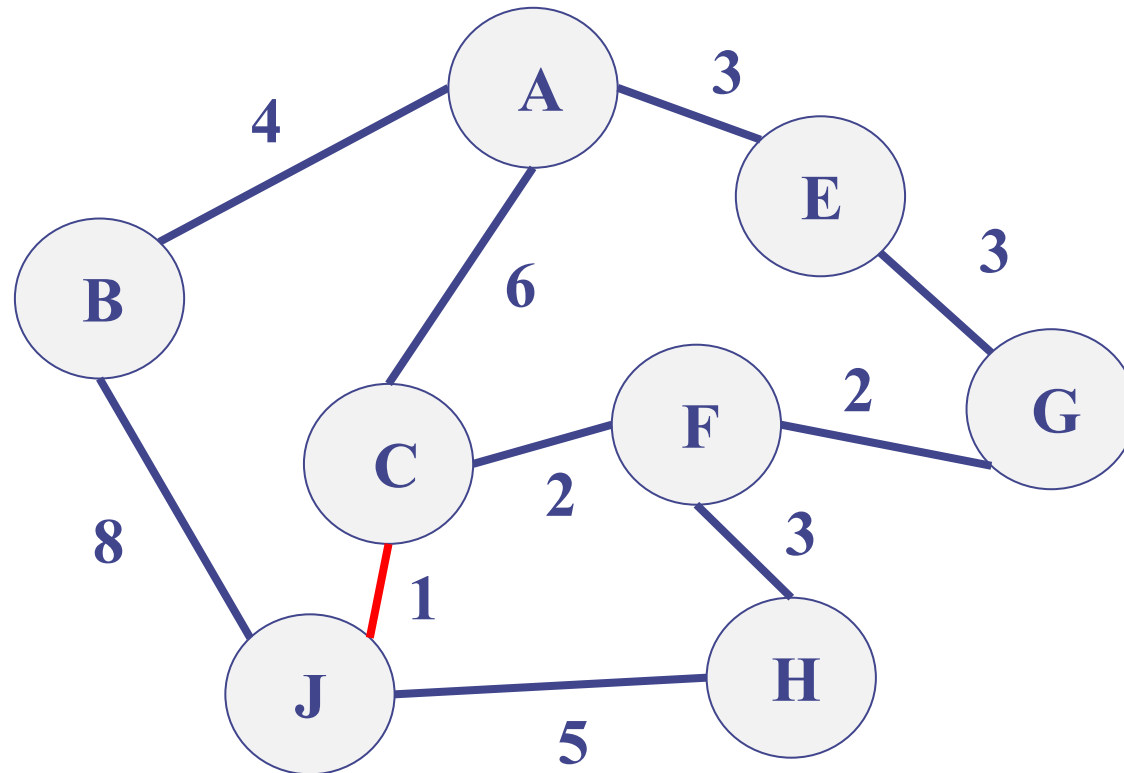
- Suppose that we **start from node C**
- Tree-So-Far (TSF) = {C}
- Options for new edges(weights) are CJ(1), CF(2), CA(6)
- Hence picked the smallest – added edge CJ(1)

Prim's Algorithm: Incorrect reasoning



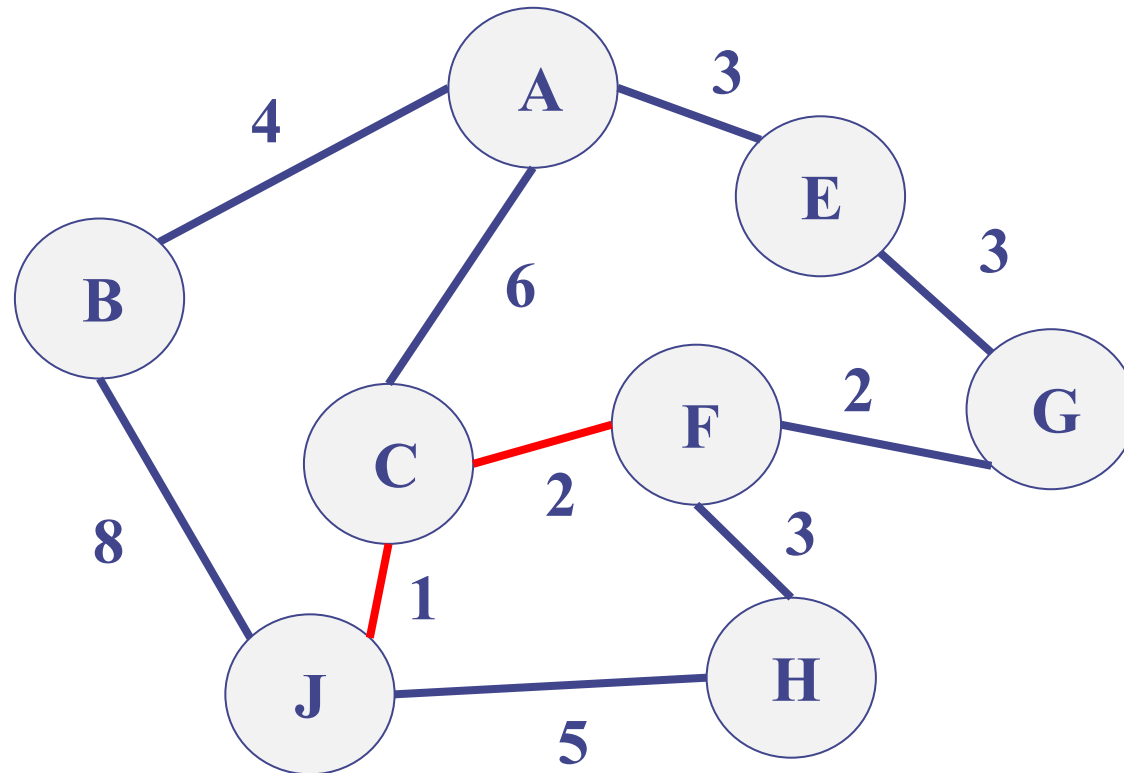
- Although tempting, it is totally wrong to think
 - “We have a path C- $\rightarrow$ J and want to extend the path, and so the only options are JB(8), JH(5)” (WRONG!)
 - Instead at each stage we just think to extend the tree, and can do so using any edge attached to the tree and that reaches a new node

Prim's Algorithm: Example (2)



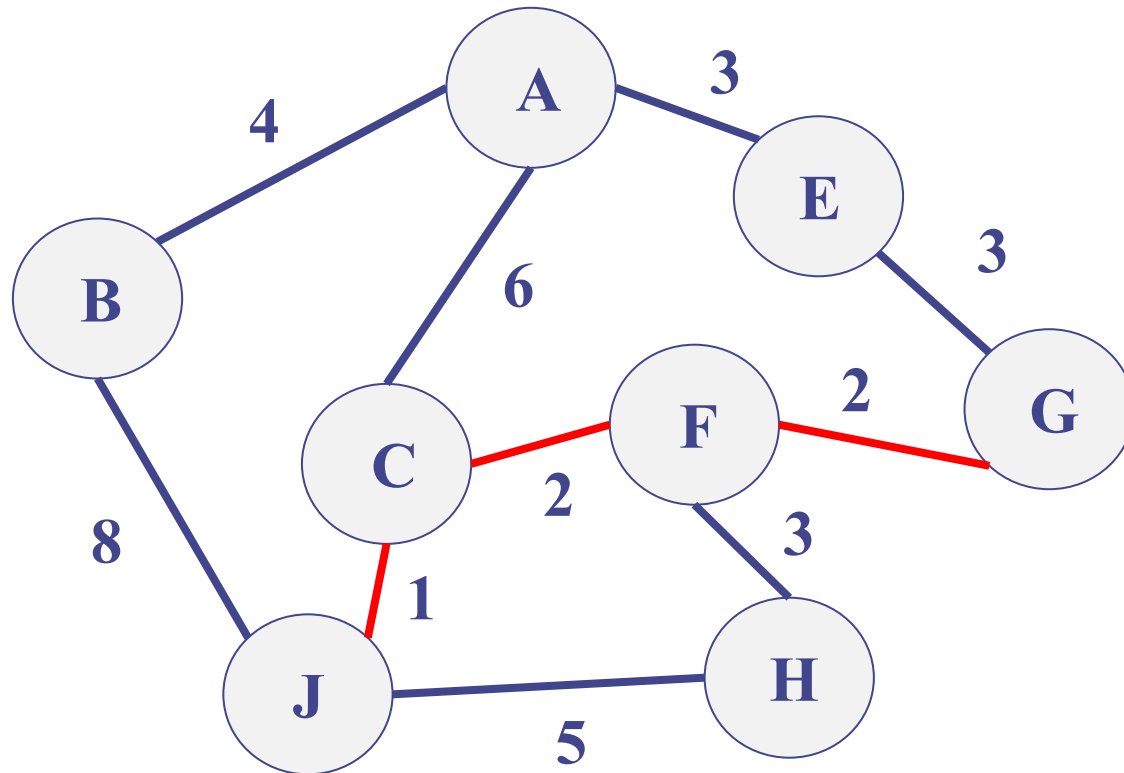
- Tree-So-Far (TSF) = {CJ(1)}
- Options for new edges(weights) are JB(8), JH(5), CF(2), CA(6)
- Hence pick the smallest – add edge CF(2)

Prim's Algorithm: Example (3)



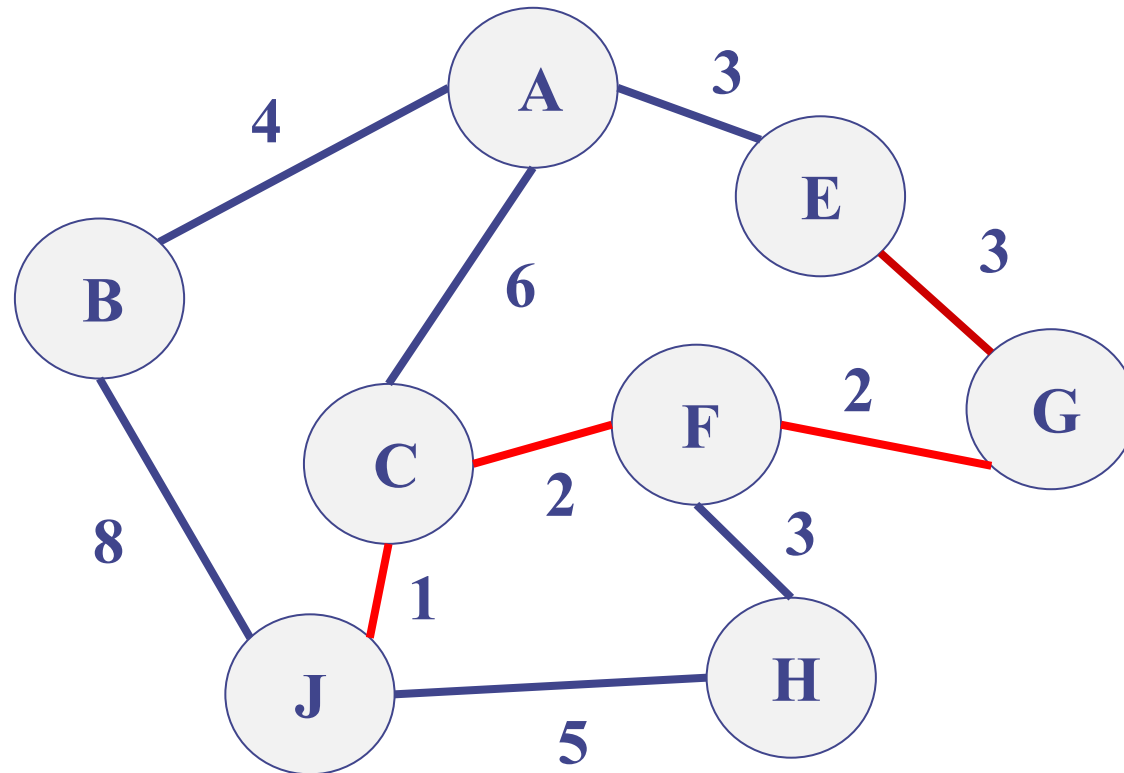
- Tree-So-Far (TSF) = {CJ,CF}
- Options for new edges(weights) are JB(8), JH(5), FG(2), FH(3), CA(6)
- Hence pick the smallest – add edge FG(2)

Prim's Algorithm: Example (4)



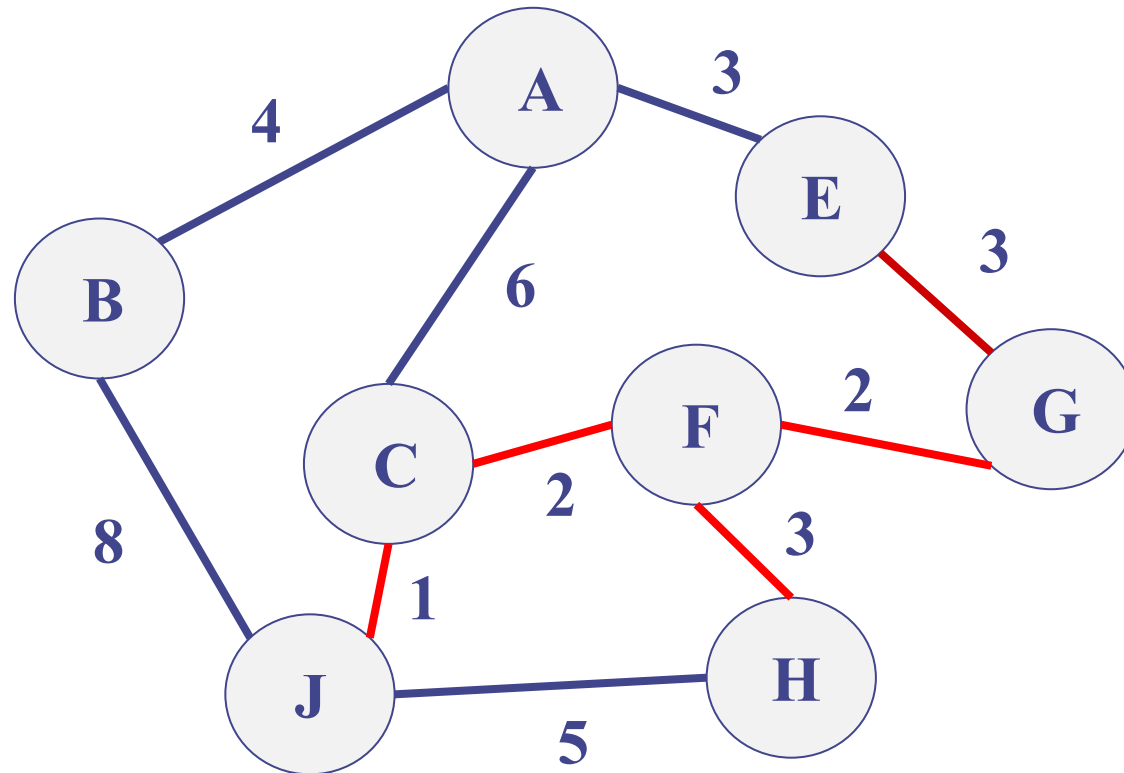
- Tree-So-Far (TSF) = {CJ,CF,FG}
- Possible new edges: JB(8), JH(5), GE(3), FH(3), CA(6)
- Now there a tie for the smallest edge: GE and FH
 - We are free to break ties in any way we want – it can be totally arbitrary or random, or any other rule desired
 - E.g. Pick GE(3)

Prim's Algorithm: Example (5)



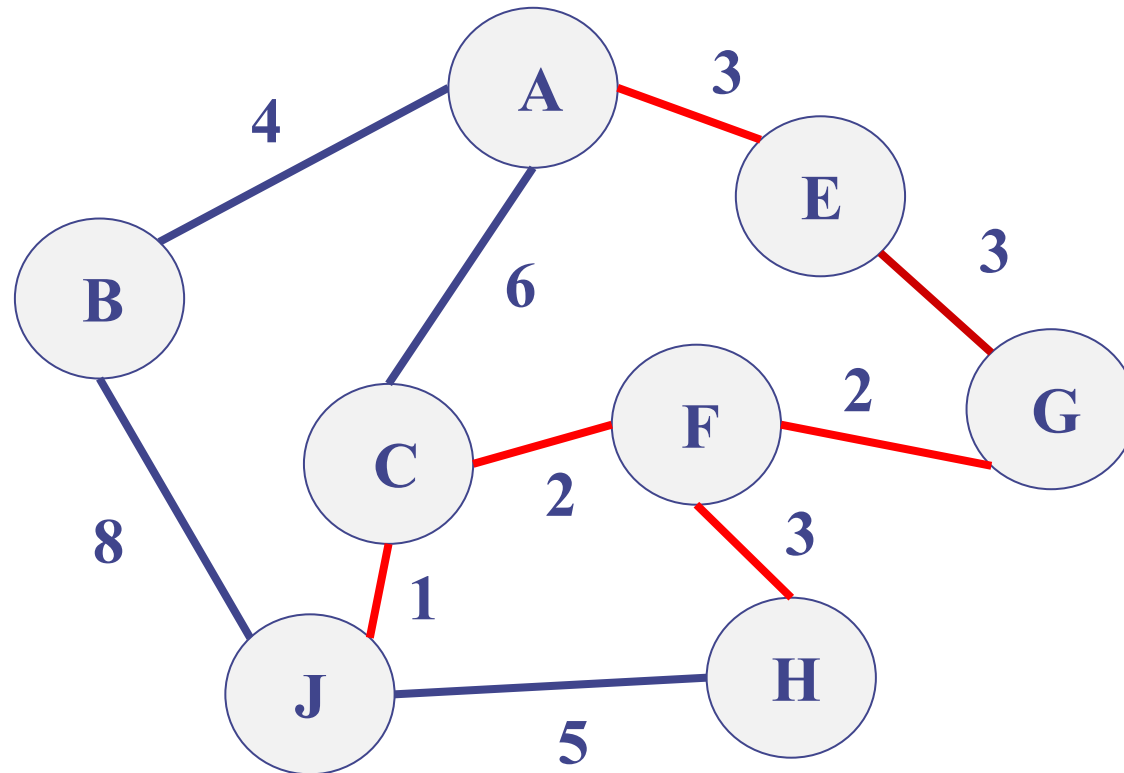
- Tree-So-Far (TSF) = {CJ,CF,FG,GE}
- Possible new edges: JB(8), JH(5), FH(3), CA(6), EA(3)
- Tie for the smallest edge: AE and FH
 - E.g. Pick FH(3)

Prim's Algorithm: Example (6)



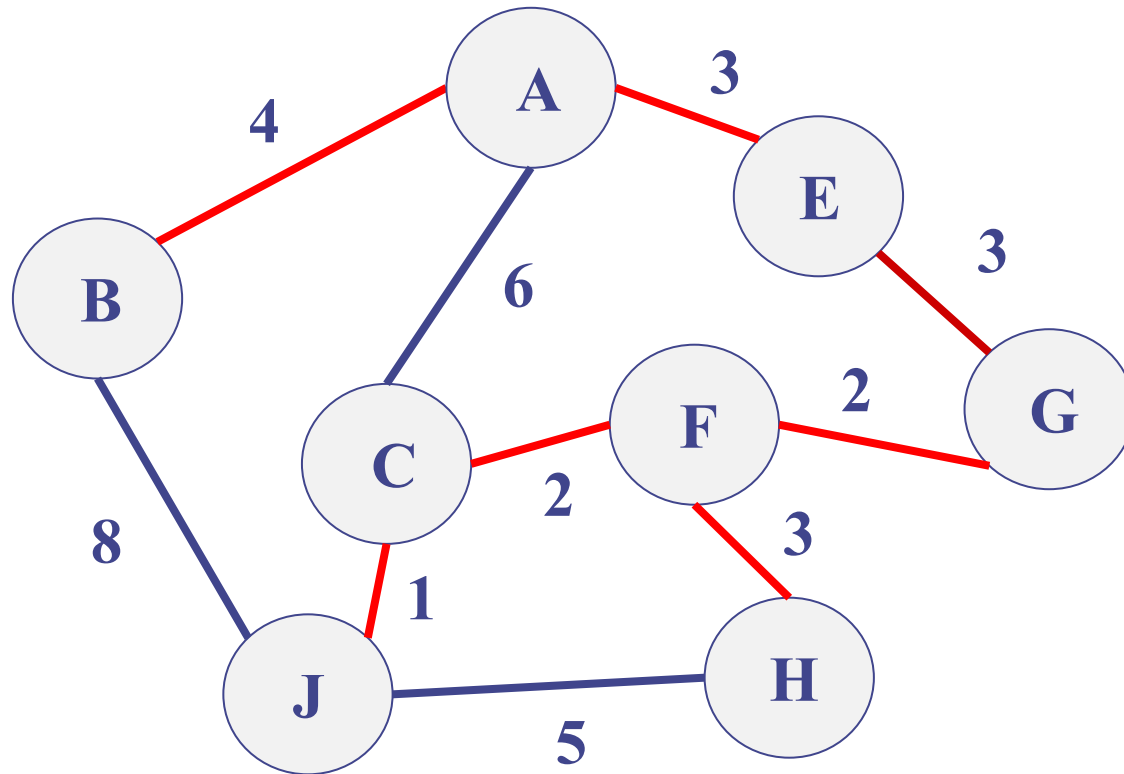
- Tree-So-Far (TSF) = {CJ,CF,FG,GE,FH}
- Possible new edges: JB(8), CA(6), EA(3)
- Have to pick EA(3)

Prim's Algorithm: Example (7)



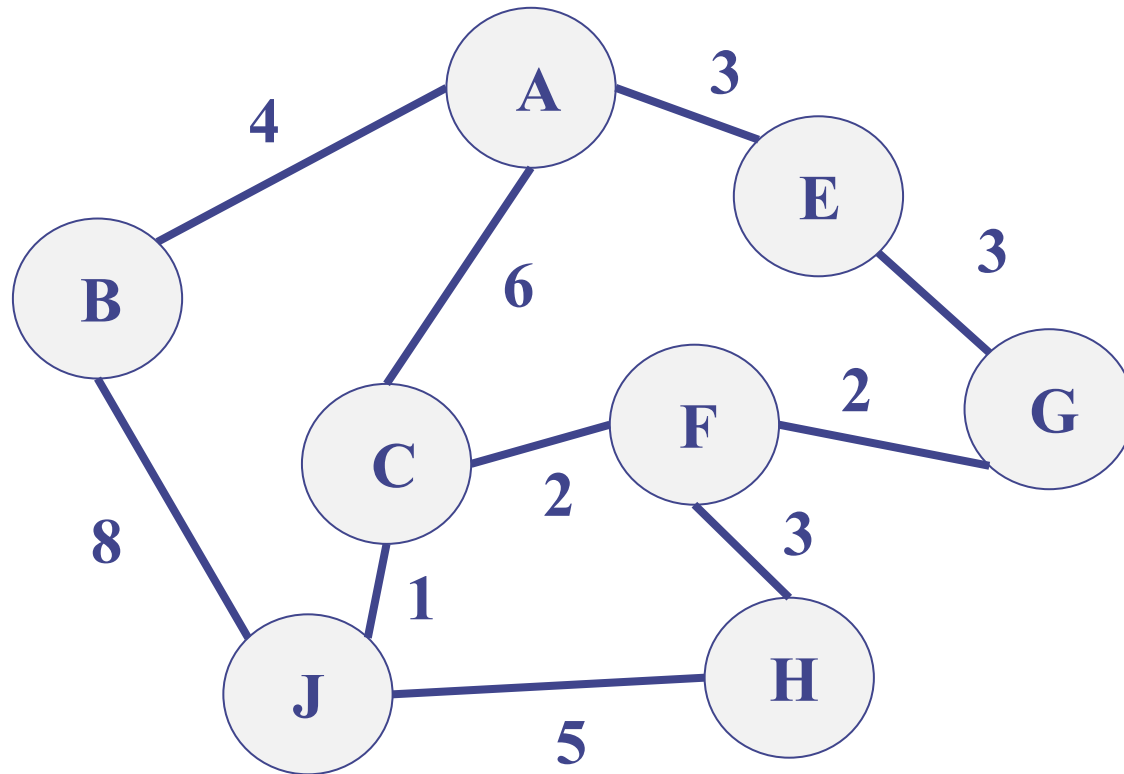
- Tree-So-Far (TSF) = {CJ,CF,FG,GE,FH,EA}
- Possible new edges: JB(8), AB(4)
- Have to pick AB(4)

Prim's Algorithm: Example (8)



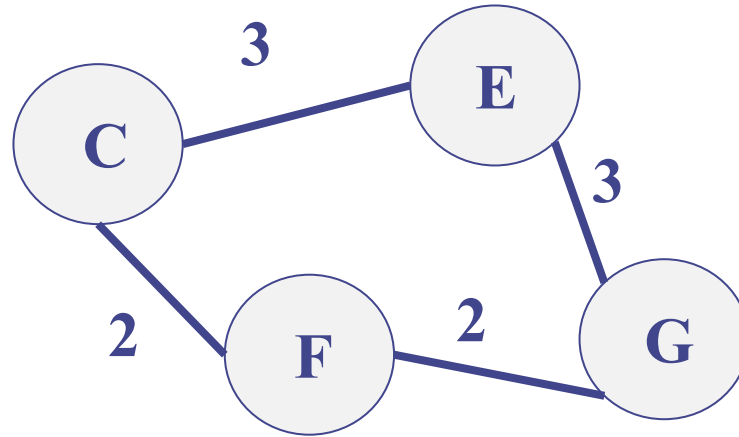
- Tree-So-Far (TSF) = {CJ,CF,FG,GE,FH,EA}
- Possible new edges: JB(8), AB(4)
- Have to pick AB
- Final spanning tree: {CJ,CF,FG,GE,FH,EA,AB}
- Note: it is a tree, not just a path!

Prim's Algorithm: Change start node



- We built the tree starting from C
- Exercises (offline):
 - For each other node in the tree, take it as the starting point and construct the tree; and also consider the effects of breaking the ties in different ways
 - Check they all have the same overall weight
 - In this case: Do all the options give the same tree?

Prim's Algorithm: Simpler example



- There are two MSTs.
- Both contain edges CF and FG, but then we can use either CE or EG
 - MST1 {CF(2),FG(2),CE(3)} weight= 7
 - MST2 {CF(2),FG(2),GE(3)} weight= 7
- Exercise: run Prim's algorithm and see that it is possible to produce either of these MSTs
 - it just depends on the ways the ties are broken

Greedy algorithm

- Prim's algorithm for constructing a Minimal Spanning Tree is a ***greedy algorithm***:
 - it just adds a minimum weight edge
 - without worrying about the overall structure, without looking ahead.
 - It makes a locally optimal choice at each step.
- However, it still gives a globally optimal answer
 - This is not typical
 - Usually, greedy methods give answers that are "good but possibly sub-optimal" (e.g. change-giving)

[assessed] WARNING

Do not confuse

- MST: Prim's to **extend a tree**
 - This module
 - Guaranteed to give an optimal result
- TSP: "Nearest Neighbour" to **extend a path**
 - E.g. AI Methods module
 - Not guaranteed to give an optimal result

They look similar but are different!!

This is a common error on assessments

Other MST Algorithms (not assessed)

- Kruskal

https://en.wikipedia.org/wiki/Kruskal%27s_algorithm

- Reverse-delete

https://en.wikipedia.org/wiki/Reverse-delete_algorithm

Minimal Expectations

- Clearly know, understand and be able to use
 - Definition of an MST
 - Prim's Algorithm to create one
 - How to apply the algorithm on graphs
 - Why it gives an optimal (minimum weight) spanning tree

Why is Prim's Optimal??

- NOT ASSESSED !!
- Just for interest

[not-assessed] Prim's Algorithm always gives an MST?

- Can “Add smallest edge” ever be a fatal mistake?
- I.e. can it be true that extending the tree using a smallest edge can make it impossible to then construct an MST?
 - Maybe, this can be a “fatal mistake” i.e. be a choice that rules out finding an MST
 - After all, we did see “greedy” could fail in the “min coins change giving”

[not-assessed] Prim's always gives an MST?

- Suppose that we have some “MST so far” T
- Define
 - $V1$: set of nodes in T
 - The nodes that have been spanned so far
 - Prim's starts with $V1 =$ some arbitrary node
 - $V2$: the set of nodes not in T
 - The nodes that we still need to include in spanning tree
 - At the last stage this is empty
- We extend the Tree-So-Far by picking an edge that connects $V1$ and $V2$
 - Need to select an edge from the set of edges $E12$ that go from any node in $V1$ to any node in $V2$

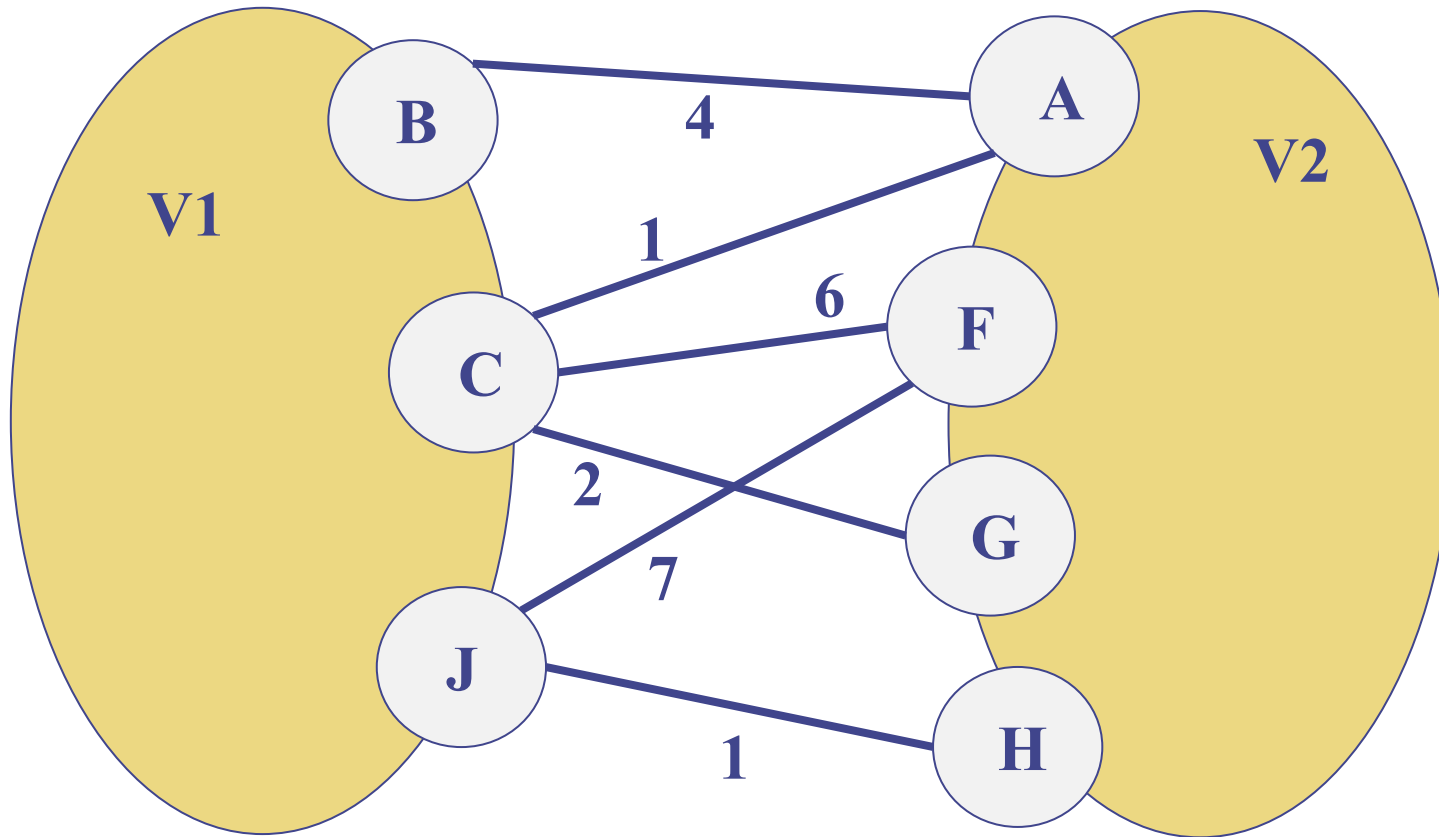
[not-assessed] Why does Prim's give an MST!?

- [Goodrich/Tamassia Textbook]. Proposition 13.25
(Section 13.7, or similar, depending on edition)
 - *"Let G be a weighted connected graph, and
 - let $V1$ and $V2$ be a partition of the vertices of G into two disjoint non-empty sets.
 - Furthermore, let e be an edge with minimum weight from among those with one endpoint in $V1$ and the other in $V2$.
 - There is an MST that has e as one of its edges."*

[not-assessed] Prop 13.25 and Prims

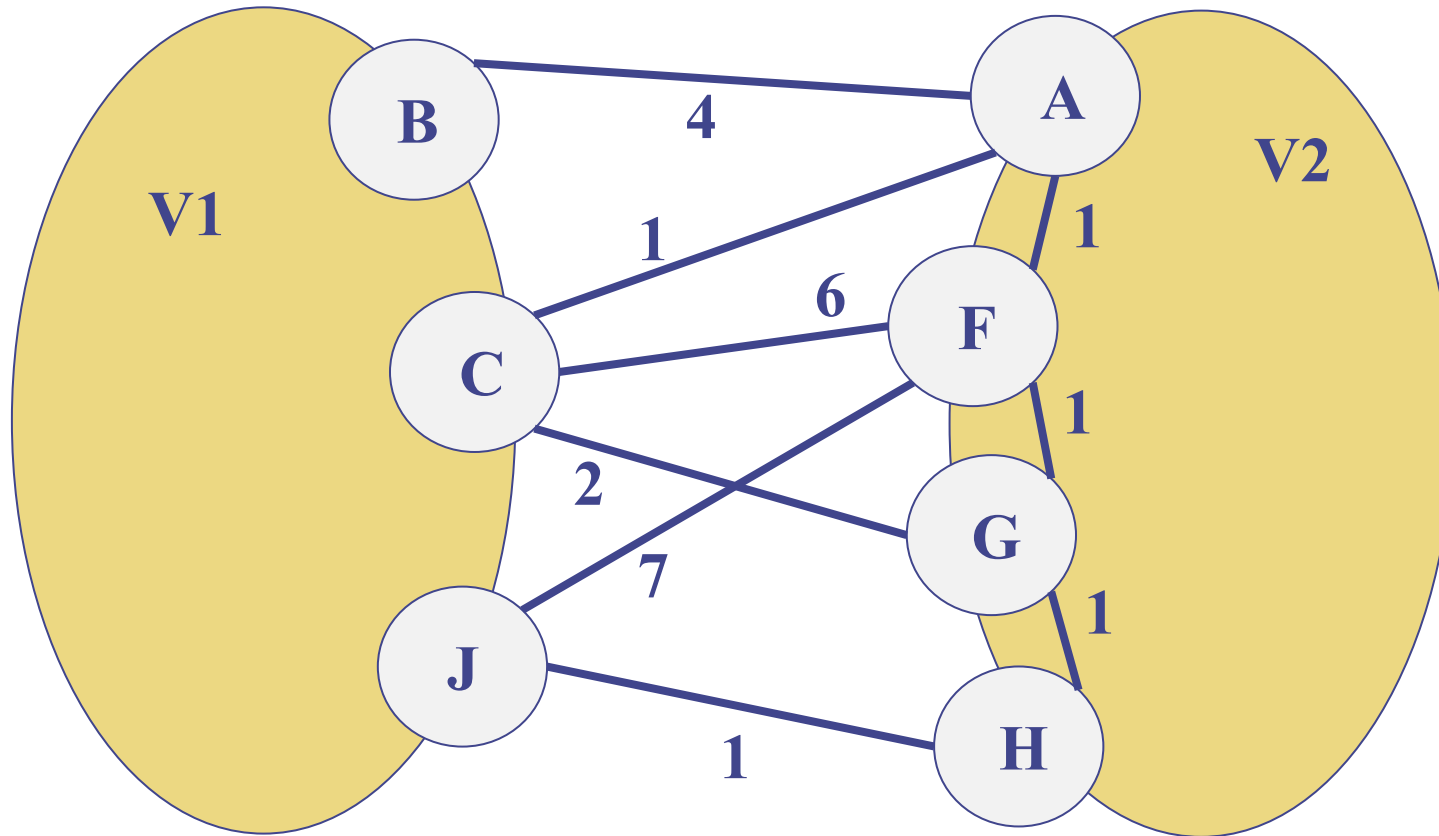
- At each stage:
 - $V1$ = vertices within the current MST
 - $V2$ = “the rest” (vertices not yet in the MST)
 - The algorithm adds a minimum weight edge between $V1$ and $V2$, and so this edge must be part of some MST
 - Hence, the construction cannot make a “fatal mistake” – at no point can it add an edge not part of an MST
- However, for more insight, and a justification, consider an example:

Prim's Algorithm: Middle stage



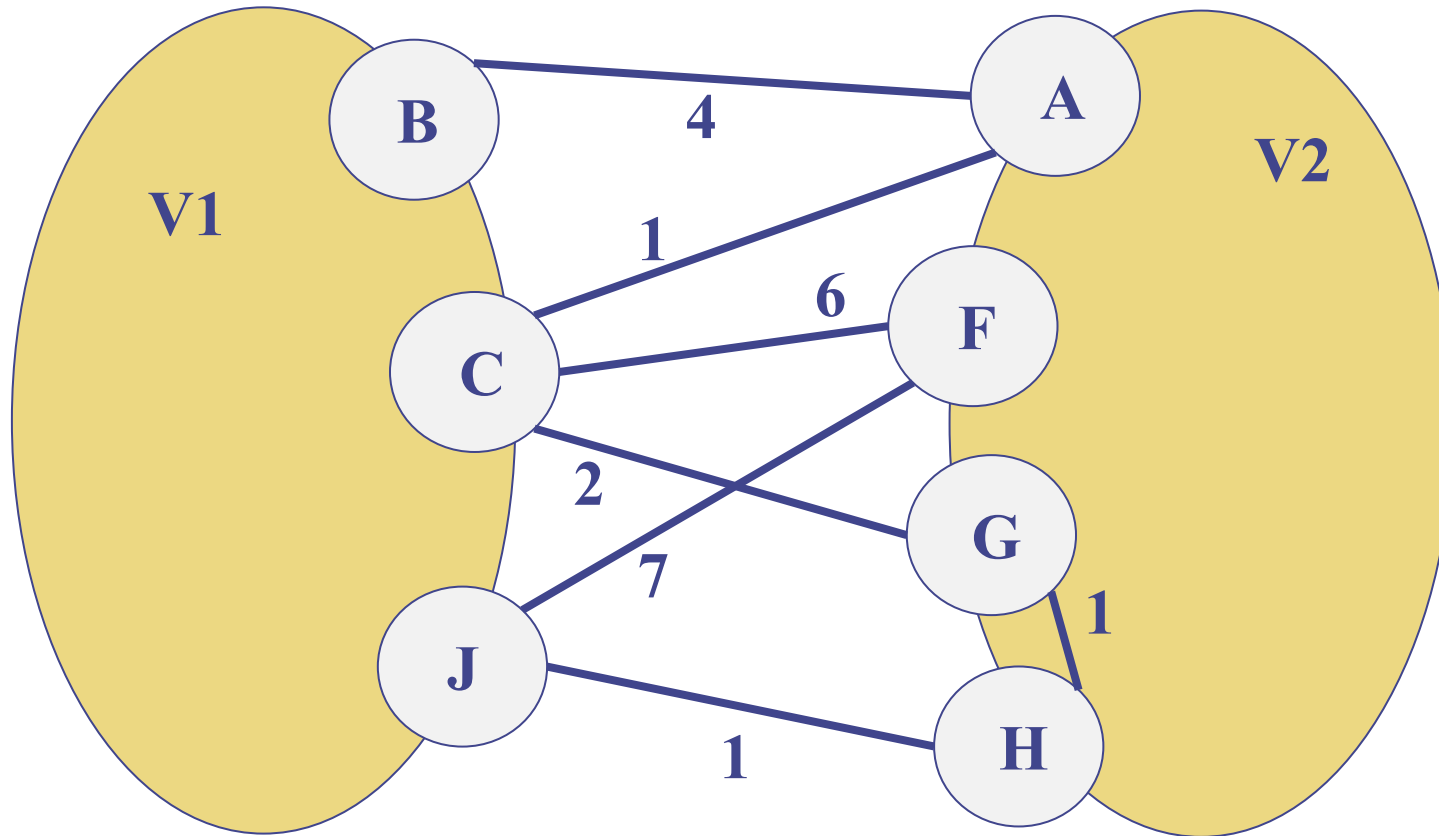
- Within V1 we already have built a tree, but need to extend it to span V2
 - For clarity, we only show the edges (and their nodes) that connect V1 and V2
- **Can it ever be a fatal mistake to pick JH – a minimum edge - to extend the tree outside of V1 ?**

Prim's Algorithm: Middle stage (2)



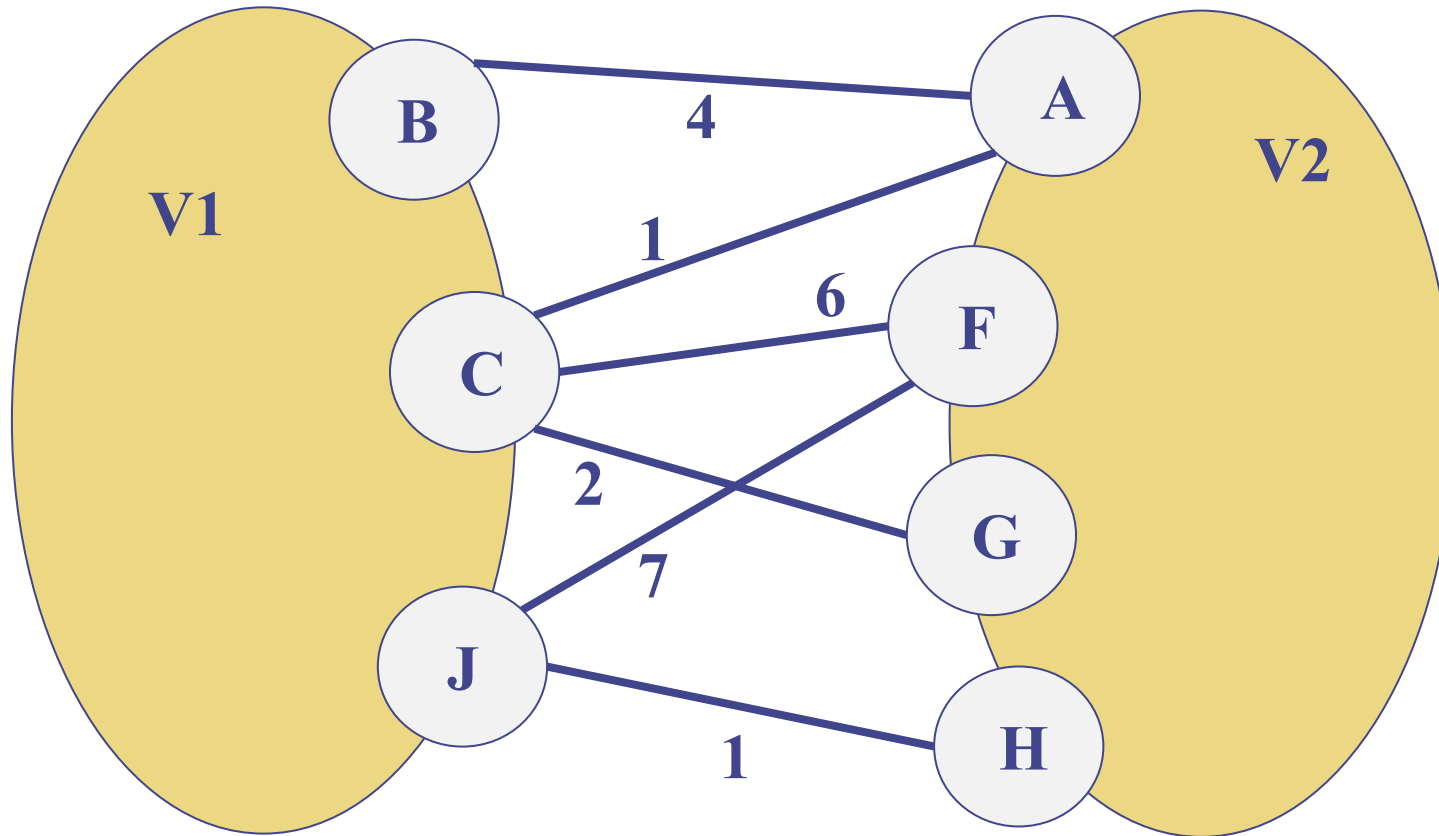
- **Can it ever be a fatal mistake to pick JH ?**
- If we picked a larger edge, such as CF(6) then could swap to JH(1) and still connect, CF(6) could be a fatal mistake.
 - E.g. suppose that A,F,G,H were all connected inside V1 with edges of weight 1, and there were no other nodes, then could swap CF back to JH and reduce the cost – so here a tree with CF could not be an MST

Prim's Algorithm: Middle stage (3)



- But note that picking CF might not be a mistake
- E.g. if F were not connected to any other nodes in V2, then we would have to use CF eventually
- Picking CF(6) might be a fatal mistake but also might not – depending on the details of the connections inside V2

Prim's Algorithm: Middle stage (4)

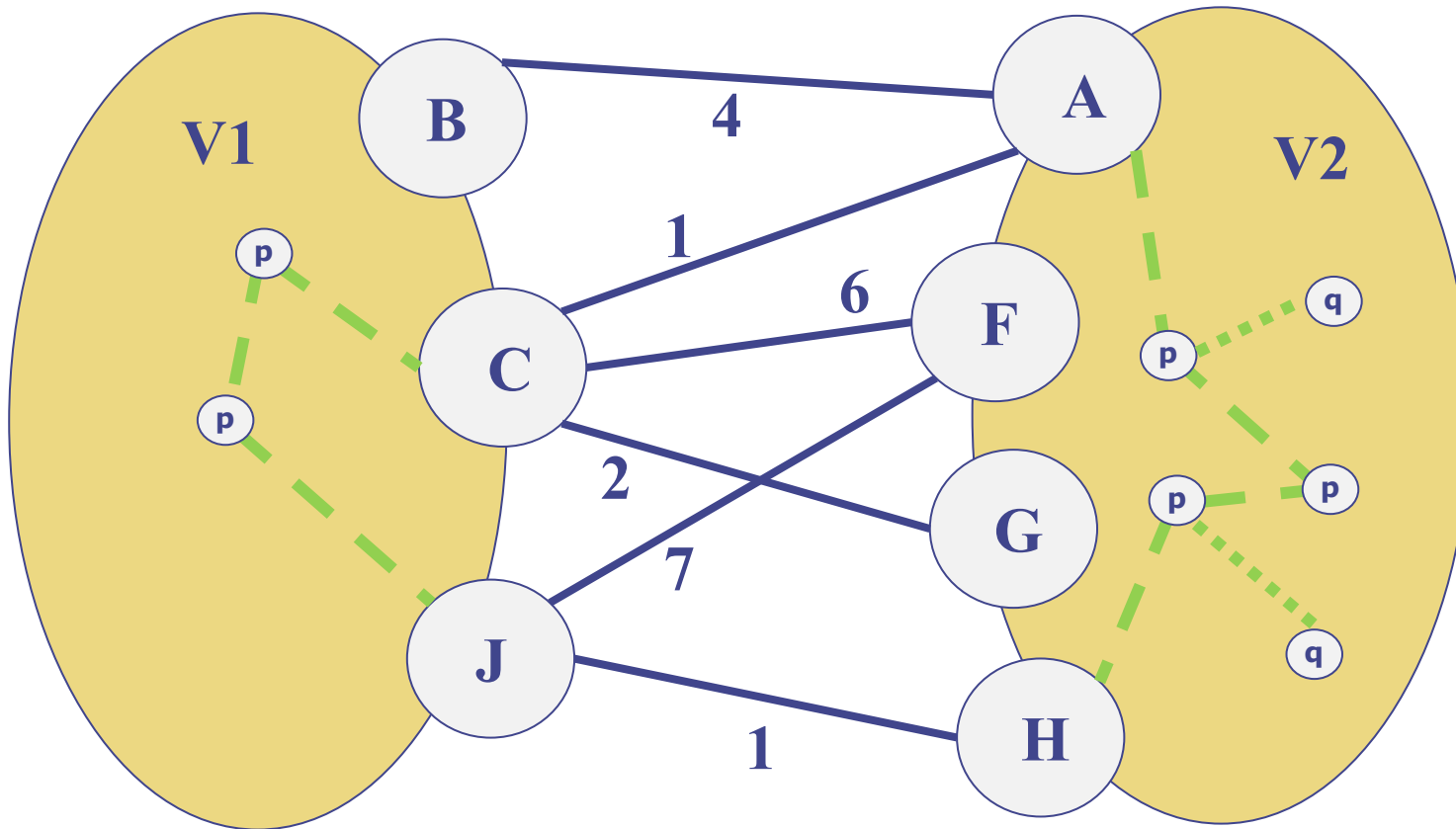


- Not picking a minimum edge can be a fatal mistake
 - (though it does not have to be a mistake)
- But can picking an arbitrary minimum edge be a mistake?
 - E.g. **is it ever possible that picking JH would inevitably lead to a non-minimum spanning tree, and we should instead pick CA ?**

Justification

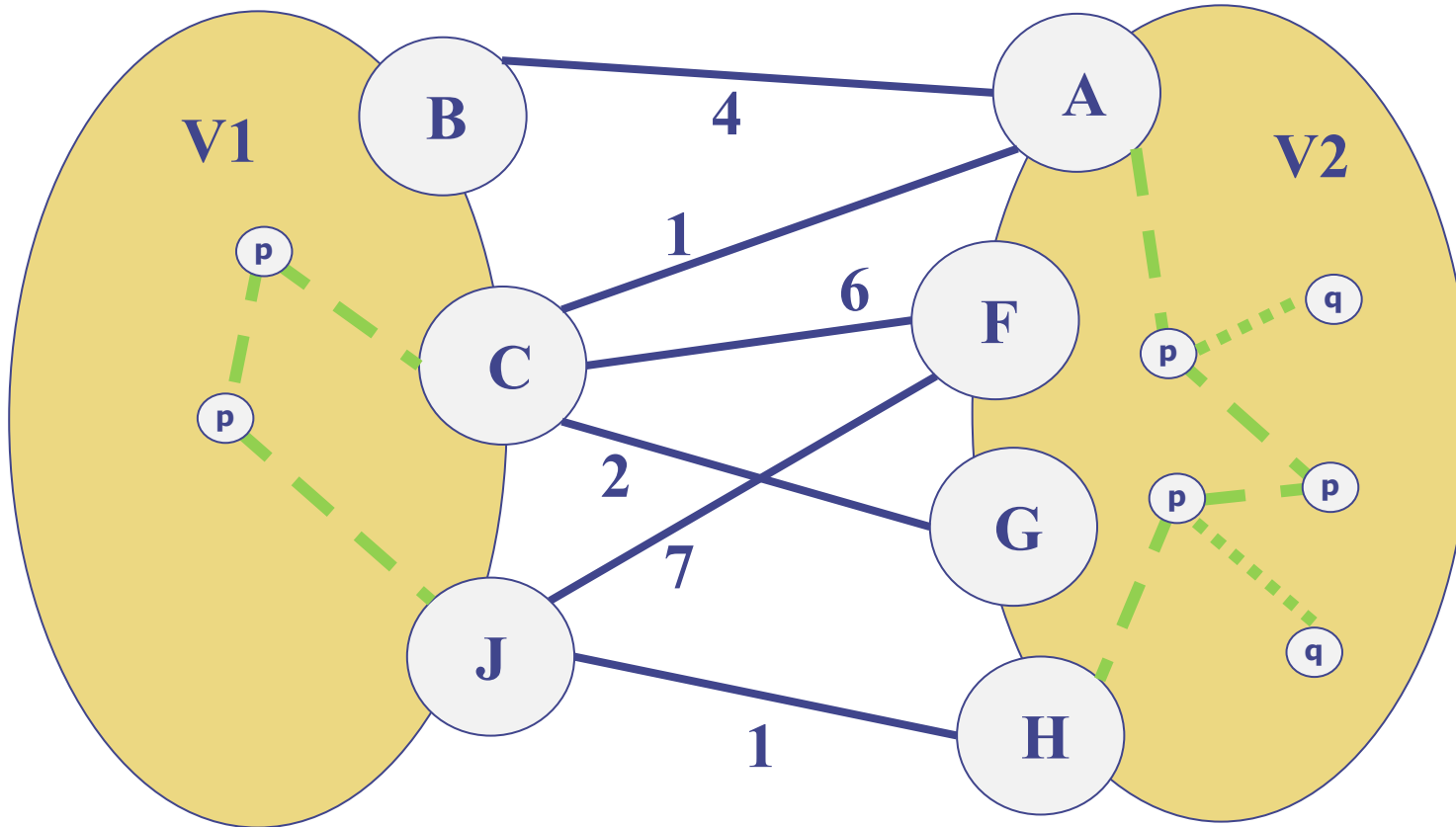
- In the style of an induction proof
 - Assume that we have not made a fatal mistake so far
 - Hence, there is some MST, call it T_1 , of the whole graph and that is consistent with the choices made in V_1 .
- If T_1 uses JH , then selecting JH is not a fatal mistake, so only need consider the case:
- MST, T_1 , did not use the edge JH
 - We will argue that T_1 can always be modified to give a different MST that uses JH :

Prim's Algorithm: Middle step



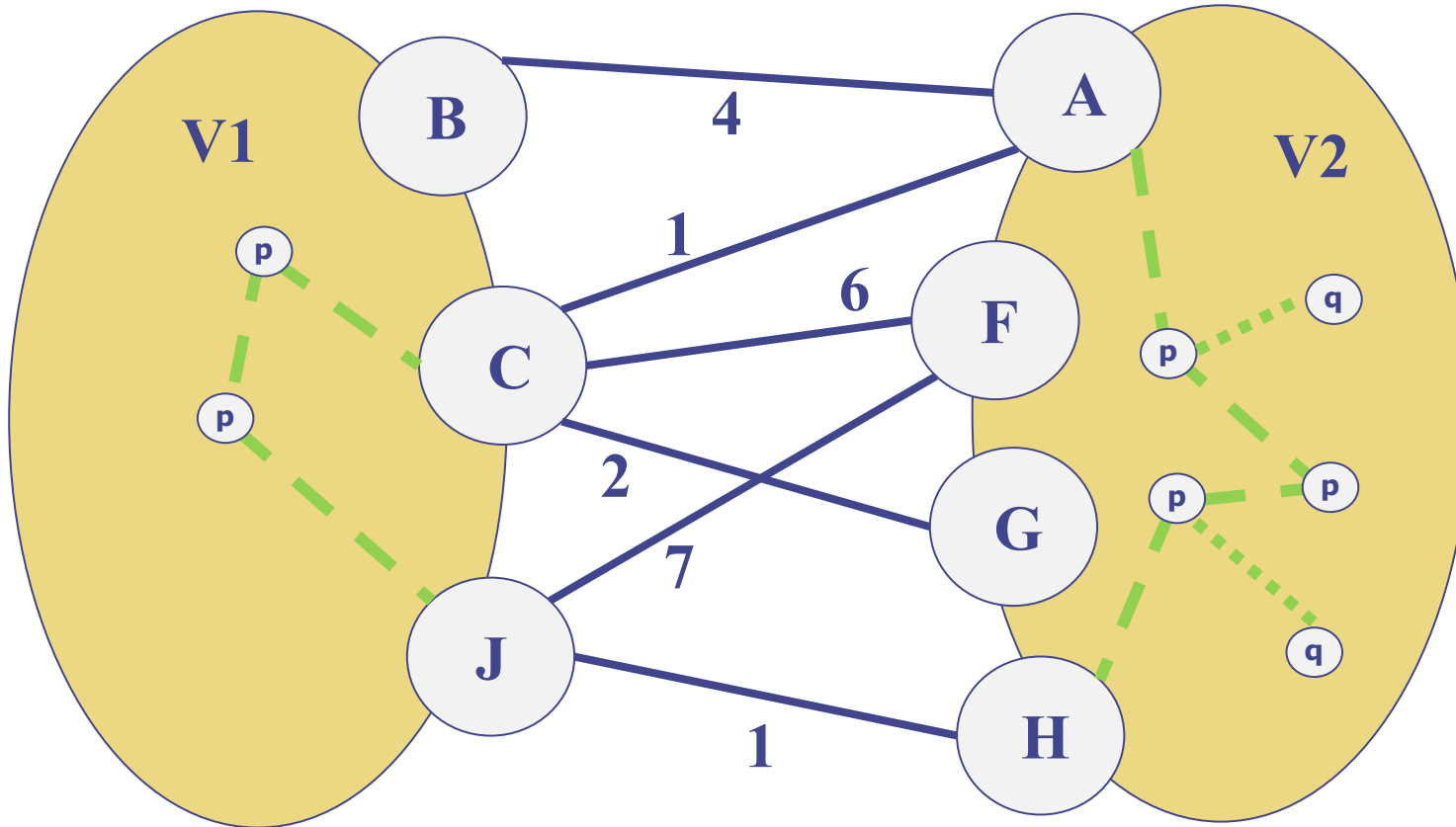
- The MST T1 does not use JH, but it is a spanning tree, and so must reach H
- So, there must be some unique path, p, in T1 from J to H
- This path must cross between V1 and V2 – in this case by CA
- Claim: In T1 we can remove CA and add edge JH, and get a new MST

Prim's Algorithm: Middle step (2)



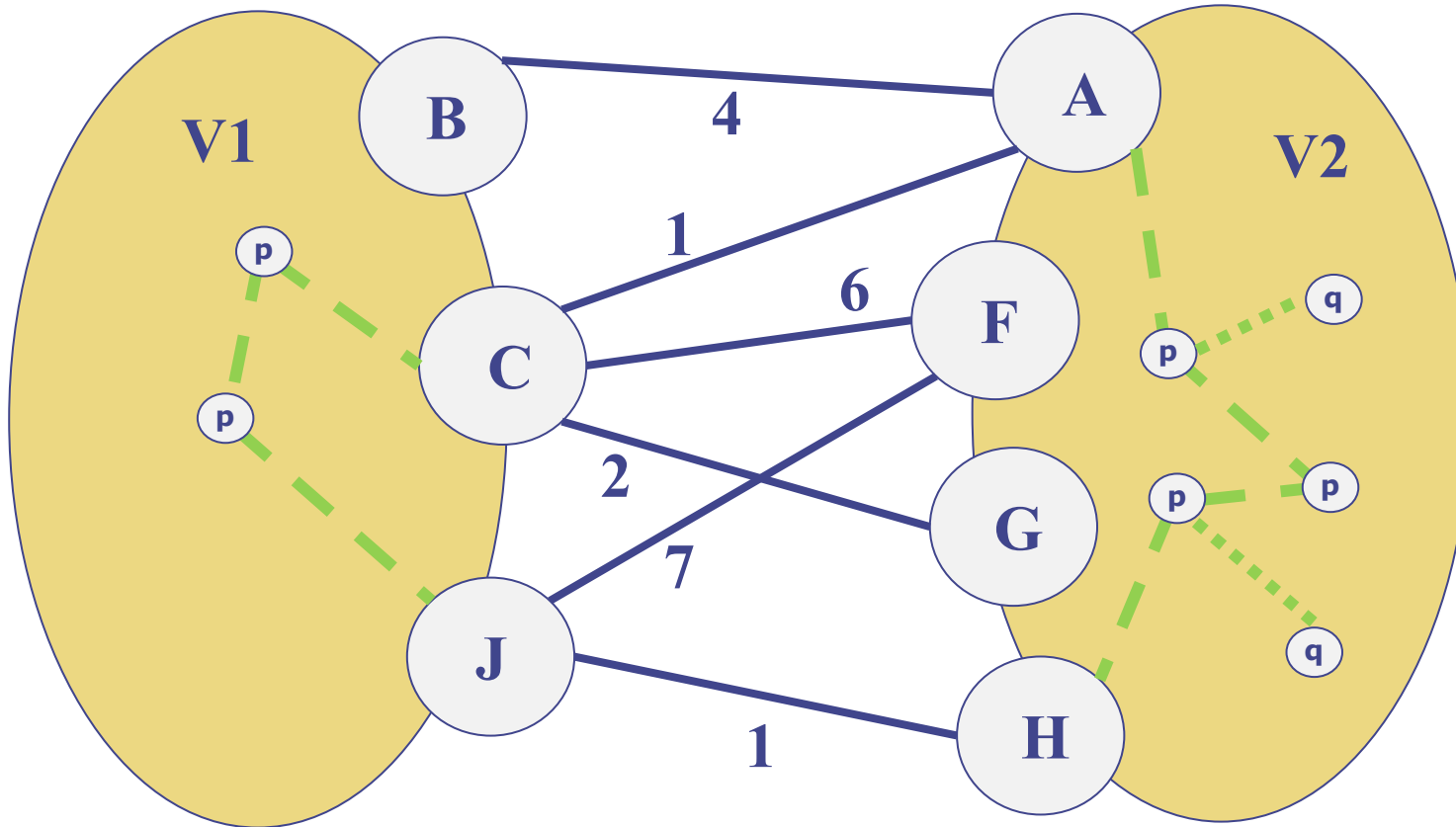
- Claim: In T1 removing CA and adding JH, gives a new MST, T2:
- Part 1: **The swap still produces a spanning tree**
- In V2 all the nodes reachable from the old edge "C to A" are still reachable, but via "J to H".
 - E.g. the "p" and "q" nodes within V2

Prim's Algorithm: Middle step (3)



- Claim: In MST T_1 , removing CA and adding JH , gives a new MST, T_2 :
- Part 2: The swap to T_2 still gives a **minimum** spanning tree
- The overall weight can not have increased from that of T_1 , as JH was a min edge and cannot be heavier than CA , so T_2 must be a separate MST

[not-assessed] Prim's Algorithm: Middle step (4)



- Conclusion: In this example, and in general, even if there are multiple minimum-weight edges crossing from V1 to V2, then an arbitrary choice is allowed between them